

Interview Questions: React Native Lists & Rendering

This guide provides 50 curated questions and answers to help you master the nuances of rendering data in React Native, ranging from foundational concepts to high-level performance tuning.

Phase 1: Core Concepts (ScrollView vs. FlatList)

- 1. What is the primary difference between ScrollView and FlatList?** ScrollView renders all child components immediately upon mounting, which is memory-intensive for large datasets. FlatList is virtualized; it only renders items currently visible on the screen (plus a small buffer), making it significantly more efficient for large lists. Refer to **BadList.tsx** for an example of why ScrollView fails with large data.
- 2. When should you choose ScrollView over FlatList?** Use ScrollView for small, fixed amounts of content where the total number of items is low (e.g., a settings page or a short form). Use FlatList for dynamic or large datasets where performance and memory management are critical.
- 3. What does "Virtualization" mean in the context of React Native lists?** Virtualization is the process of maintaining a limited window of rendered items. As the user scrolls, items moving out of the "window" are unmounted to reclaim memory, and new items moving into view are mounted.
- 4. Can you nest a FlatList inside a ScrollView with the same orientation?** Technically yes, but it is highly discouraged. Doing so breaks the FlatList virtualization logic because the ScrollView provides an infinite height/width, forcing the FlatList to render all items at once.
- 5. What is the purpose of the data prop in FlatList?** The data prop accepts a simple array of data that the list will iterate over to create its rows.
- 6. How do you implement a horizontal list in React Native?** By setting the horizontal prop to true on either a ScrollView or a FlatList.
- 7. What is SectionList, and how does it differ from FlatList?** SectionList is used for grouped data. It supports "Sticky Headers" for sections, whereas FlatList is intended for a single, continuous stream of data.

Phase 2: Props, Keys, and rendering Logic

- 8. What is the function of the renderItem prop?** It is a callback function that takes an item from the data array and returns a JSX element. As seen in **renderItem.tsx**, it typically extracts { item, index, separators }.

9. Why is the `keyExtractor` prop important? It provides a unique string key for each item. This allows React to efficiently track which items have changed, been added, or removed, preventing unnecessary re-renders of the entire list.

10. What happens if you don't provide a `keyExtractor`? React Native will default to using the item's index as a key. This can lead to UI bugs and performance issues if the list order changes or items are deleted.

11. Why should you avoid using `Math.random()` in a `keyExtractor`? Because keys must be stable. If a key changes every render, React will treat every item as "new," unmounting and remounting the entire list, which destroys performance.

12. What is the `extraData` prop used for? `FlatList` is a `PureComponent`, meaning it only re-renders if its props change. If your `renderItem` depends on state outside of the data prop (like a "selected" ID), you must pass that state to `extraData` to signal a re-render.

13. How do you handle an empty list state? Use the `ListEmptyComponent` prop. This allows you to render a specific view (like "No items found") when the data array is empty, as demonstrated in `CompleteListExample.tsx`.

14. How can you add a search bar that scrolls away with the list? Pass the search bar component to the `ListHeaderComponent` prop. This keeps the header within the scrollable area without breaking virtualization.

15. What is the difference between `ListHeaderComponent` and `ListFooterComponent`? The header renders at the top of the list; the footer renders at the bottom. Both are useful for keeping UI elements like search bars or loading indicators within the list's scroll logic.

Phase 3: Interaction & Pagination

16. How is "Pull to Refresh" implemented in a `FlatList`? By using the `refreshControl` prop with a `RefreshControl` component, or by providing the `onRefresh` and `refreshing` props directly to the `FlatList`.

17. What is `onEndReached`? A callback triggered when the user has scrolled close to the end of the data. This is the primary hook used for implementing infinite scroll or lazy loading.

18. What does `onEndReachedThreshold` represent? It is a numeric value (e.g., `0.5`) that defines how far from the end of the list (in terms of visible screen length) the scroll must be to trigger `onEndReached`.

19. How do you prevent `onEndReached` from firing multiple times per page? Implement a guard clause using a loading state. If loading is true, return early from the fetch function. See `LazyList.tsx` for this logic.

20. How do you scroll to a specific index programmatically? Use a ref to the FlatList and call the `scrollToIndex({ index })` method.

21. What is the purpose of `initialScrollIndex`? It allows the list to start rendering from a specific item index rather than the very beginning (index 0).

22. How do you render a list in a grid format (multiple columns)? Use the `numColumns` prop (e.g., `numColumns={2}`). Note that changing this prop usually requires a full component remount.

23. What prop allows you to style the container that wraps all the list items? The `contentContainerStyle` prop. This is where you should add padding or margins to the scrollable area itself.

24. What is `ItemSeparatorComponent`? A prop that renders a component between every item in the list, but not at the very top or bottom. It is better than adding margins to items because it prevents double-spacing.

Phase 4: Performance Optimization

25. What is `getItemLayout` and why is it the "ultimate" optimization? It allows you to provide the exact dimensions (height/width) of items to the list. This lets FlatList skip the expensive process of measuring items dynamically as they scroll into view.

26. When can you NOT use `getItemLayout`? When your list items have variable or dynamic heights that cannot be calculated before rendering.

27. What is `initialNumToRender`? The number of items to render in the first batch. Setting this appropriately (usually enough to fill the screen) improves initial load perceived performance.

28. What does `windowSize control`? It determines how many items stay mounted in memory outside the visible area. The default is 10 (10 above, 10 below, 1 visible). Reducing this saves memory; increasing it reduces the "white flash" when scrolling fast.

29. What is `maxToRenderPerBatch`? It limits the number of items rendered per incremental scroll batch. Lowering this reduces UI "jank" but can increase the appearance of blank spaces during fast scrolls.

30. How does `removeClippedSubviews` improve performance? When set to true, views that are off-screen are detached from the native view hierarchy. This is particularly effective on Android for reducing memory usage.

31. Why should you use `React.memo` or `PureComponent` for list items? To prevent an item from re-rendering unless its specific data changes. In a list of 100 items, you don't want all 100 to re-render just because the parent component's state updated.

32. What is the "White Flash" issue in FlatList? It occurs when a user scrolls faster than the engine can render new items, showing the blank background of the list container.

33. How do you fix the "White Flash"? By increasing `windowSize`, optimizing the `renderItem` component, or implementing `getItemLayout`.

34. Is it better to use an anonymous function inside `renderItem`? No. Defining the function inside the prop (e.g., `renderItem={({item}) => ...}`) creates a new function reference on every render, which can lead to unnecessary re-renders. Define it as a static function or a class method.

35. What is `updateCellsBatchingPeriod`? The time delay (in milliseconds) between rendering batches. It allows the JS thread to handle other tasks (like touch events) between rendering rows.

Phase 5: Advanced Scenarios & Troubleshooting

36. How do you implement "Sticky Headers" in a FlatList? Use the `stickyHeaderIndices` prop, providing an array of the indices you want to stick to the top (e.g., `[0]`).

37. How do you handle "Inverted" lists (like a chat app)? Set the `inverted` prop to `true`. This flips the scroll direction and starts the list at the bottom.

38. What is the `onViewableItemsChanged` prop? A callback that triggers when the set of visible items in the list changes. This is useful for tracking analytics (e.g., "was this ad seen?") or autoplaying videos.

39. What is `viewabilityConfig`? A configuration object for `onViewableItemsChanged` that defines what percentage of an item must be visible to count as "viewed."

40. Why might a list feel "heavy" on low-end devices? Usually due to complex item components, high `windowSize`, or missing `keyExtractor` and `getItemLayout` optimizations.

41. How do you handle images in a large list to prevent crashes? Use a library like `react-native-fast-image` for caching and better memory management, as standard `<Image>` components can easily trigger Out of Memory (OOM) errors in large lists.

42. How do you implement a "Load More" button instead of infinite scroll? Put a `TouchableOpacity` in the `ListFooterComponent` that triggers your fetch function when pressed.

43. Can you use FlatList for a grid where items have different heights? Standard `FlatList` does not support "Masonry" layouts well. For that, you would typically use a community library like `@shopify/flash-list` or `react-native-masonry-list`.

44. What is `@shopify/flash-list`? A high-performance alternative to `FlatList` that recycles views more aggressively and is often significantly faster, especially on Android.

45. What happens if your `keyExtractor` returns non-unique values? React Native will likely throw a warning, and you will see "ghost" items, duplicate components, or items jumping around during scrolls.

46. How do you style the space between columns in a multi-column list? Apply horizontal margins to the item component and use `columnWrapperStyle` on the `FlatList` to align the items (e.g., `justifyContent: 'space-between'`).

47. What is `maintainVisibleContentPosition`? A prop (currently experimental/iOS mostly) that helps maintain the user's scroll position when new content is prepended to the top of the list (very useful for chat apps).

48. How do you debug `FlatList` performance? Use the "Show Perf Monitor" in the debug menu to watch the "JS" and "UI" frame rates. Use `console.log` inside `renderItem` to see if items are re-rendering too often.

49. What is the purpose of `ListHeaderComponentStyle`? It allows you to apply styles specifically to the container wrapping the `ListHeaderComponent`.

50. What is the most common reason for a `FlatList` not rendering at all? Usually, the parent container does not have a defined height (it needs `flex: 1`) or the `data` prop is being passed an undefined value instead of an empty array.